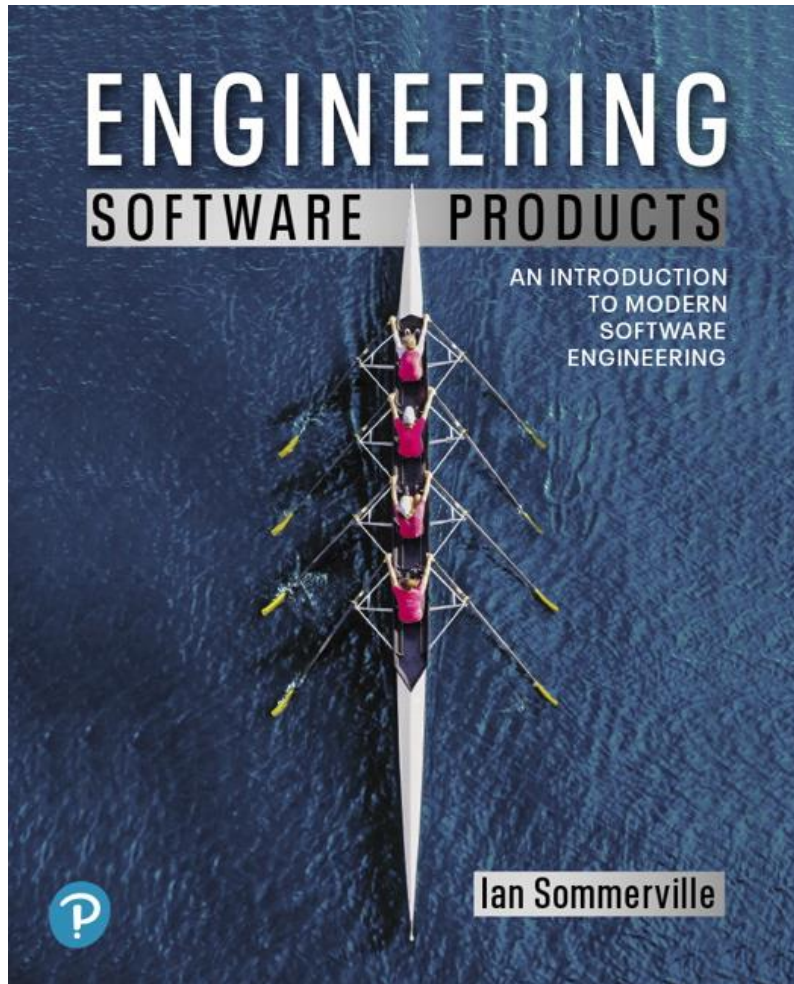


Engineering Software Products

First Edition



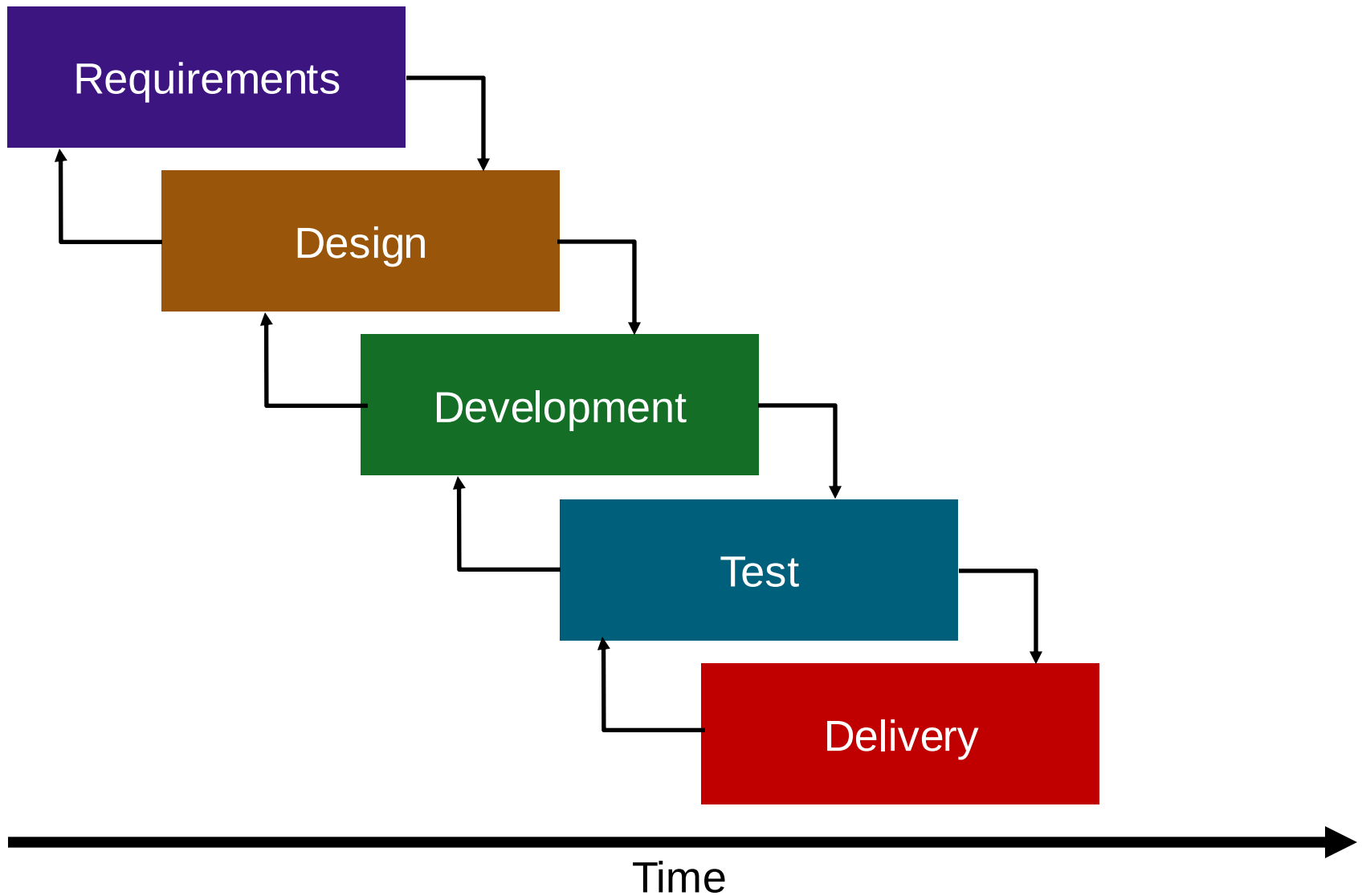
Chapter 2

Agile Software Engineering

Plan-Driven Development Process

- ‘Waterfall’ process
 - Strong project planning
 - Requirements collection
 - Formal development and QA
- Large teams, complex projects
- Significant overhead in planning
- Lots of documentation
- *Describe the kind of project where you think plan-driven process would be effective?*

Waterfall



Agile Definition

Able to move quickly and easily.

Agile software engineering (1 of 2)

- Software products must be brought to market quickly so rapid software development and delivery is essential.
- Virtually all software products are now developed using an agile approach.
- Agile software engineering focuses on delivering functionality quickly, responding to changing product specifications and minimizing development overheads.

Agile software engineering (2 of 2)

- A large number of ‘agile methods’ have been developed.
 - There is no ‘best’ agile method or technique.
 - It depends on who is using the technique, the development team and the type of product being developed

Agile methods (1 of 2)

- Plan-driven development evolved to support the engineering of large, long-lifetime systems (such as aircraft control systems) where teams may be geographically dispersed and work on the software for several years.
 - This approach is based on controlled and rigorous software development processes that include detailed project planning, requirements specification and analysis and system modelling.
 - However, plan-driven development involves significant overheads and documentation, and it does not support the rapid development and delivery of software.

Agile methods (2 of 2)

- Agile methods were developed in the 1990s to address this problem.
 - These methods focus on the software rather than its documentation, develop software in a series of increments and aim to reduce process bureaucracy as much as possible.
 - They included Extreme Programming, SCRUM, DSDM, Adaptive Software Development, Crystal, Feature-Driven Development, Pragmatic Programming
 - February 11-13, 2001, at The Lodge at Snowbird ski resort in the Wasatch mountains of Utah, seventeen people met to talk and find common ground

AGILE

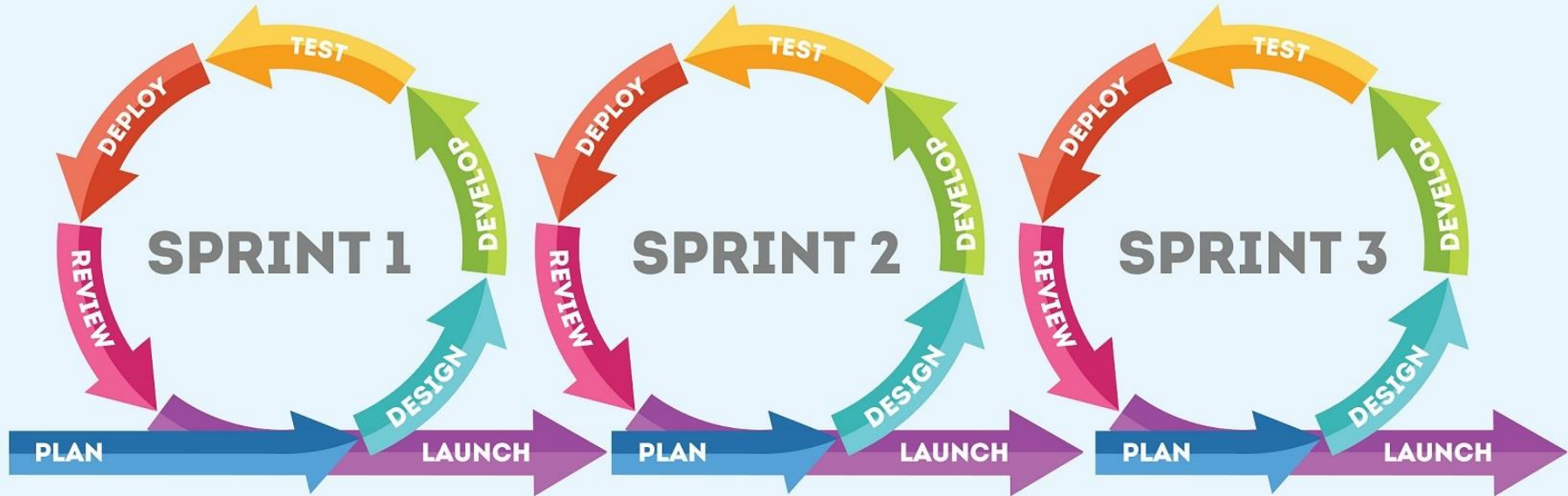


Table 2.1 The agile manifesto

- Agile values:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

**While there is value on the items on the right,
we value the items on the left more.**

<https://agilemanifesto.org/>

Principles Behind The Agile Manifesto

(1 of 4)

- Our highest priority is to **satisfy the customer through early and continuous delivery of valuable software.**
- **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
- **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Principles Behind The Agile Manifesto

(2 of 4)

- Business people and developers must **work together daily** throughout the project.
- Build projects around **motivated individuals**. Give them the environment and support they need and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.

Principles Behind The Agile Manifesto

(3 of 4)

- **Working software** is the primary measure of progress.
- Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous **attention to technical excellence** and good design enhances agility.

Principles Behind The Agile Manifesto

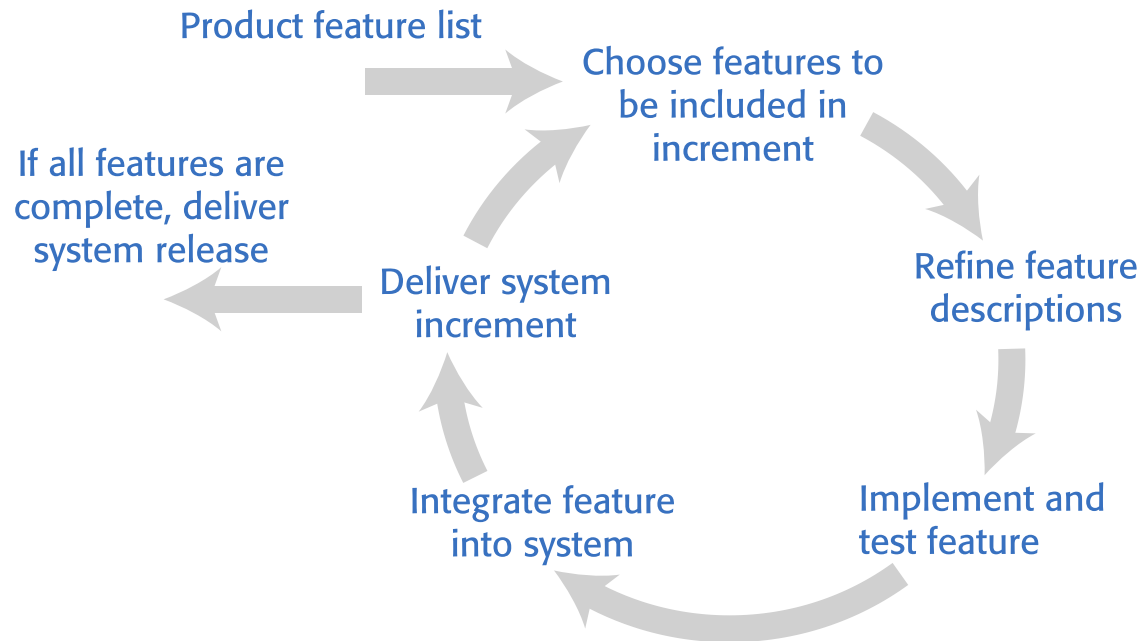
(4 of 4)

- **Simplicity**--the art of maximizing the amount of work not done--is essential.
- The best architectures, requirements, and designs emerge from **self-organizing teams**.
- At regular intervals, the **team reflects** on how to become more effective, then tunes and adjusts its behavior accordingly.

Incremental development

- All agile methods are based around incremental development and delivery.
- Product development focuses on the software features, where a feature does something for the software user.
- With incremental development, you start by prioritizing the features so that the most important features are implemented first.
 - You only define the details of the feature being implemented in an increment.
 - That feature is then implemented and delivered.
- Users or surrogate users can try it out and provide feedback to the development team. You then go on to define and implement the next feature of the system.

Figure 2.1



Incremental development

Table 2.2 Incremental development activities (1 of 2)

Activity	Description
Choose features to be included in an Increment	Using the list of features in the planned product, select those features that can be implemented in the next product increment.
Refine feature descriptions	Add detail to the feature descriptions so that the team members have a common understanding of each feature and there is sufficient detail to begin implementation.
Implement and test	Implement the feature and develop automated tests for that feature that show that its behavior is consistent with its description. I explain automated testing in Chapter 9.

Table 2.2 Incremental development activities (2 of 2)

Activity	Description
Integrate feature and test	Integrate the developed feature with the existing system and test it to check that it works in conjunction with other features.
Deliver system increment	Deliver the system increment to the customer or product manager for checking and comments. If enough features have been implemented, release a version of the system for customer use.

Extreme programming (1 of 2)

- The most influential work that has changed software development culture was the development of Extreme Programming (XP).
- The name was coined by Kent Beck in 1998 because the approach was developed by pushing recognized good practice, such as iterative development, to 'extreme' levels.
- Extreme programming focused on 12 new development techniques that were geared to rapid, incremental software development, change and delivery.

Extreme programming (2 of 2)

- Some of these techniques are now widely used; others have been less popular.
- The most widely used XP techniques (highlighted in red on the following slide) are explained elsewhere in the book.

Figure 2.2

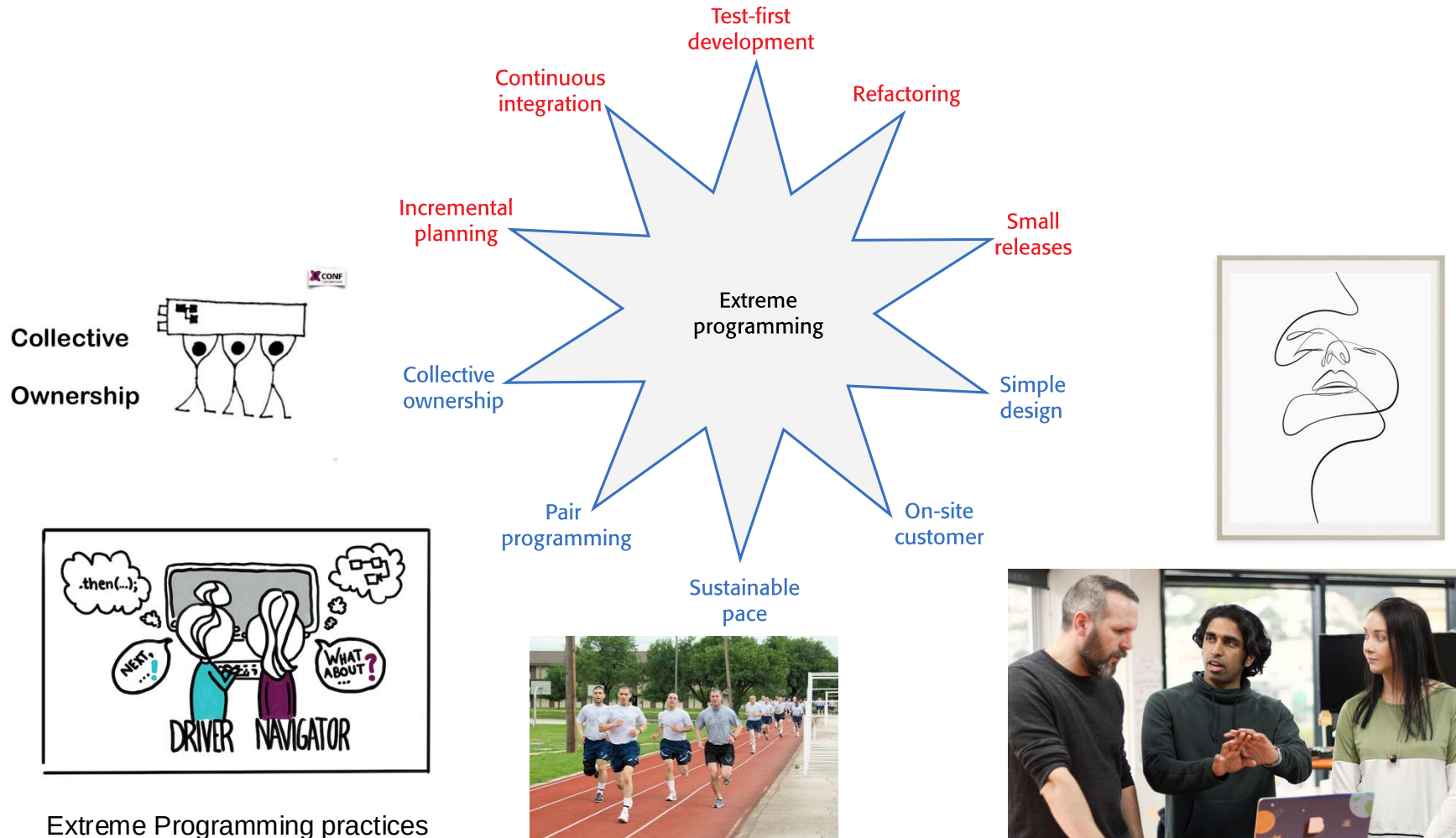


Table 2.4 Widely adopted XP practices

(1 of 2)

Practice	Description
Incremental planning/ user Stories	There is no “grand plan” for the system. Instead, what needs to be implemented (the requirements) in each increment are established in discussions with a customer representative. The requirements are written as user stories. The stories to be included in a release are determined by the time available and their relative priority.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the previous release.
Test-driven development	Instead of writing code and then tests for that code, developers write the tests first. This helps clarify what the code should actually do and that there is always a “tested” version of the code available. An automated unit test framework is used to run the tests after every change. New code should not “break” code that has already been implemented.

Table 2.4 Widely adopted XP practices

(2 of 2)

Practice	Description
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system and a new version of the system is created. All unit tests from all developers are run automatically and must be successful before the new version of the system is accepted.
Refactoring	Refactoring means improving the structure, readability, efficiency, and security of a program. All developers are expected to refactor the code as soon as potential code improvements are found. This keeps the code simple and maintainable.

Scrum

- Software company managers need information that will help them understand how much it costs to develop a software product, how long it will take and when the product can be brought to market.
- Plan-driven development provides this information through long-term development plans that identify deliverables - items the team will deliver and when these will be delivered.
- Plans always change so anything apart from short-term plans are unreliable.
- Scrum is an agile method that provides a framework for agile project organization and planning. It does not mandate any specific technical practices.

Table 2.5 Scrum terminology (1 of 2)

Scrum term	Explanation
Product	The software product that is being developed by the Scrum team.
Product Owner	A team member who is responsible for identifying product features and attributes. The Product Owner reviews work done and helps to test the product.
Product backlog	A to-do list of items such as bugs, features, and product improvements that the Scrum team has not yet completed.
Development team	A small self-organizing team of five to eight people who are responsible for developing the product.
Sprint	A short period, typically two to four weeks, when a product increment is developed.
Scrum	A daily team meeting where progress is reviewed and work to be done that day is discussed and agreed.

Table 2.5 Scrum terminology (2 of 2)

Scrum term	Explanation
ScrumMaster	A team coach who guides the team in the effective use of Scrum.
Potentially shippable product increment	The output of a sprint that is of high enough quality to be deployed for customer use.
Velocity	An estimate of how much work a team can do in a single sprint.

Key roles in Scrum (1 of 2)

- ***The Product Owner*** is responsible for ensuring that the development team are always focused on the product they are building rather than diverted into technically interesting but less relevant work.
 - In product development, the product manager should normally take on the Product Owner role.

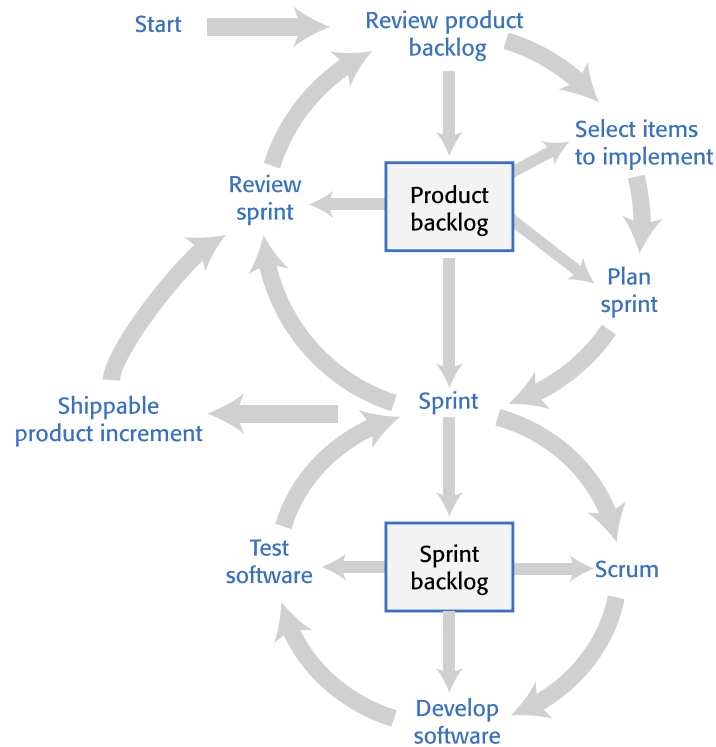
Key roles in Scrum (2 of 2)

- ***The ScrumMaster*** is a Scrum expert whose job is to guide the team in the effective use of the Scrum method. The developers of Scrum emphasize that the ScrumMaster is not a conventional project manager but is a coach for the team. They have authority within the team on how Scrum is used.
 - In many companies that use Scrum, the ScrumMaster also has some project management responsibilities.

Scrum and sprints

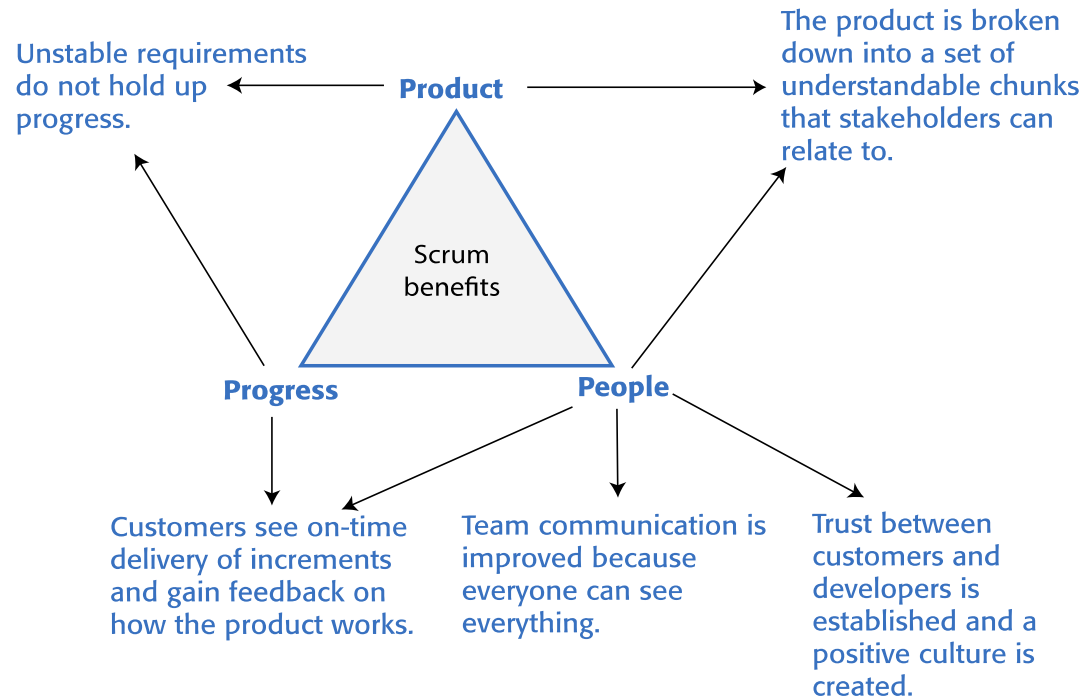
- In Scrum, software is developed in sprints, which are fixed-length periods (2 - 4 weeks) in which software features are developed and delivered.
- During a sprint, the team has daily meetings (Scrums) to review progress and to update the list of work items that are incomplete.
- Sprints should produce a 'shippable product increment'. This means that the developed software should be complete and ready to deploy.

Figure 2.3



Scrum cycle

Figure 2.4



The top five benefits of using Scrum

Key Scrum practices

- ***Product backlog***

This is a to-do list of items to be implemented that is reviewed and updated before each sprint.

- ***Timeboxed sprints***

Fixed-time (2-4 week) periods in which items from the product backlog are implemented,

- ***Self-organizing teams***

Self-organizing teams make their own decisions and work by discussing issues and making decisions by consensus.

Product backlogs

- The product backlog is a list of what needs to be done to complete the development of the product.
- The items on this list are called product backlog items (PBIs).
- The product backlog may include a variety of different items such as product features to be implemented, user requests, essential development activities and desirable engineering improvements.
- The product backlog should always be prioritized so that the items that be implemented first are at the top of the list.

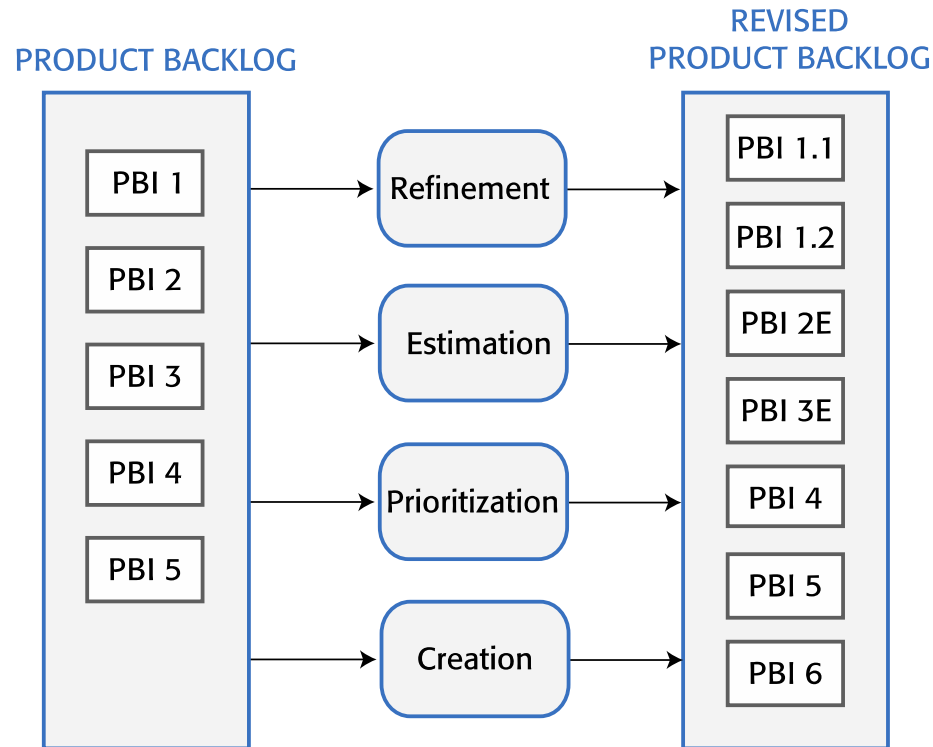
Table 2.6 Examples of product backlog items

-
1. As a teacher, I want to be able to configure the group of tools that are available to individual classes. (feature)
 2. As a parent, I want to be able to view my children's work and the assessments made by their teachers. (feature)
 3. As a teacher of young children, I want a pictorial interface for children with limited reading ability. (user request)
 4. Establish criteria for the assessment of open source software that might be used as a basis for parts of this system. (development activity)
 5. Refactor user interface code to improve understandability and performance. (engineering improvement)
 6. Implement encryption for all personal user data. (engineering improvement)
-

Table 2.7 Product backlog item states

Heading	Description
Ready for consideration	These are high-level ideas and feature descriptions that will be considered for inclusion in the product. They are tentative so may radically change or may not be included in the final product.
Ready for refinement	The team has agreed that this is an important item that should be implemented as part of the current development. There is a reasonably clear definition of what is required. However, work is needed to understand and refine the item.
Ready for implementation	The PBI has enough detail for the team to estimate the effort involved and to implement the item. Dependencies on other items have been identified.

Figure 2.5



Product backlog activities

Product backlog activities (1 of 2)

- ***Refinement***

Existing PBIs are analysed and refined to create more detailed PBIs. This may lead to the creation of new product backlog items.

- **Estimation**

The team estimate the amount of work required to implement a PBI and add this assessment to each analysed PBI.

Product backlog activities (2 of 2)

- **Creation**

New items are added to the backlog. These may be new features suggested by the product manager, required feature changes, engineering improvements, or process activities such as the assessment of development tools that might be used.

- **Prioritization**

The product backlog items are reordered to take new information and changed circumstances into account.

BREAK

PBI estimation metrics (1 of 2)

- ***Effort required***

- This may be expressed in person-hours or person-days i.e. the number of hours or days it would take one person to implement that PBI. This is not the same as calendar time. Several people may work on an item, which may shorten the calendar time required.

- ***Story points***

- Story points are an arbitrary estimate of the effort involved in implementing a PBI, taking into account the size of the task, its complexity, the technology that may be required and the ‘unknown’ characteristics of the work.

PBI estimation metrics (2 of 2)

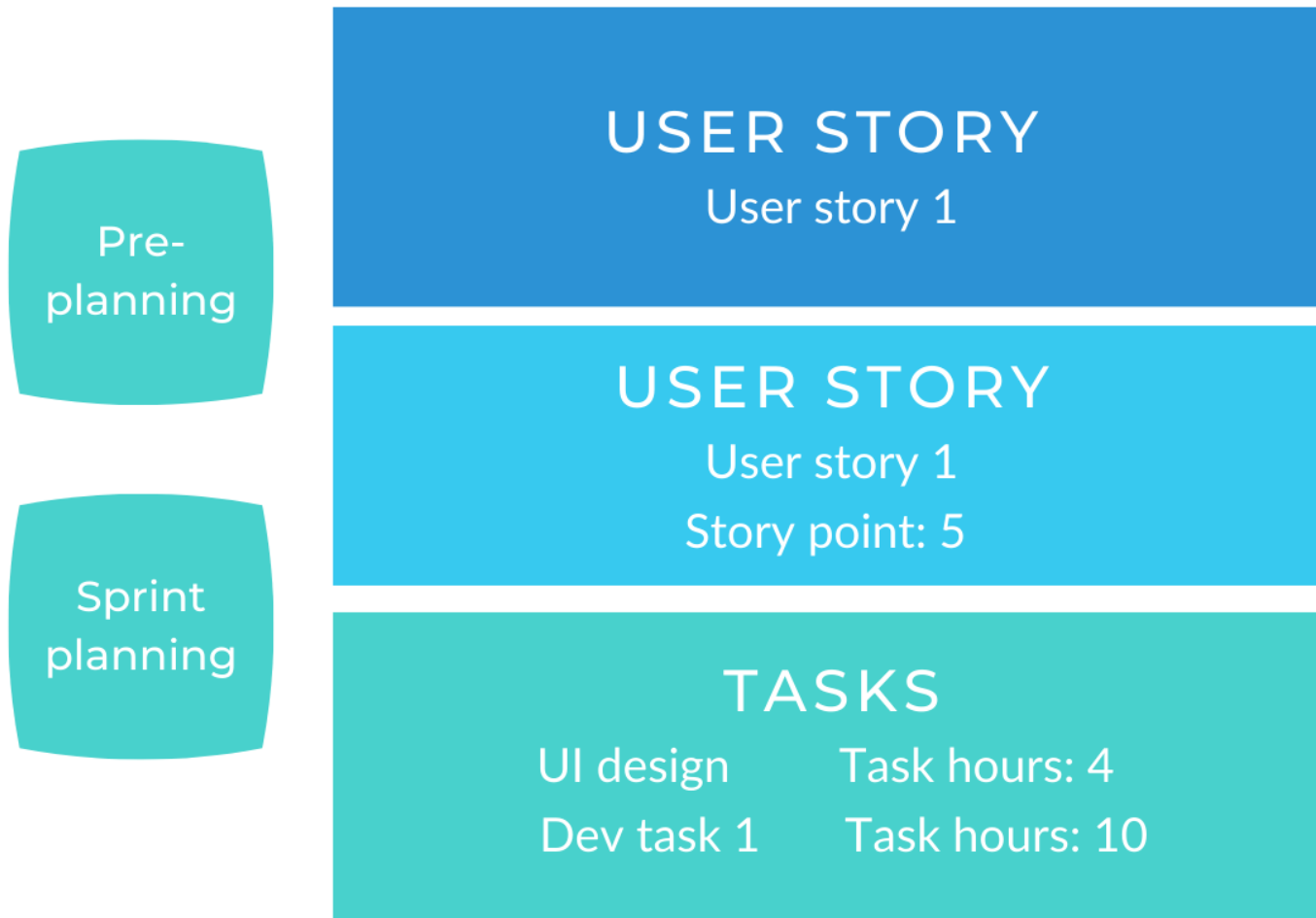
- ***Story points***

- They were derived originally by comparing user stories, but they can be used for estimating any kind of PBI.
- Story points are estimated relatively. The team agree on the story points for a baseline task and other tasks are estimated by comparison with this e.g. more/less complex, larger/smaller etc.
- Can use linear or Fibonacci sequence
 - Series of numbers with each number being the sum of the two previous numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21
 - Agile uses 0.5, 1, 2, 3, 5, 8, 13, 21

Story point estimation table

1	Refresh list button	Save filter
2	Update to issue	Duplicate metric
3	Add metric	Change metric type
5	Add button	Rearrange columns
8	Solve edit issue	Multi-project support

Story points vs. hours



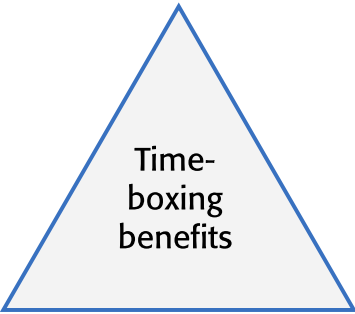
Timeboxed sprints

- Products are developed in a series of sprints, each of which delivers an increment of the product or supporting software.
- Sprints are short duration activities (1-4 weeks) and take place between a defined start and end date. Sprints are timeboxed, which means that development stops at the end of a sprint whether or not the work has been completed.
- During a sprint, the team work on the items from the product backlog.

Figure 2.6

There is a tangible output (usually a software demonstrator) that can be delivered at the end of every sprint.

Demonstrable progress



Time-
boxing
benefits

Problem discovery

If errors and omissions are discovered the rework required is limited to the duration of a sprint.

Work planning

The team develops an understanding of how much work they can do in a fixed time period.

Benefits of using timeboxed sprints

Sprint activities (1 of 2)

- ***Sprint planning***

Work items to be completed in that sprint are selected and, if necessary, refined to create a sprint backlog. This should not last more than a day at the beginning of the sprint.

- ***Sprint execution***

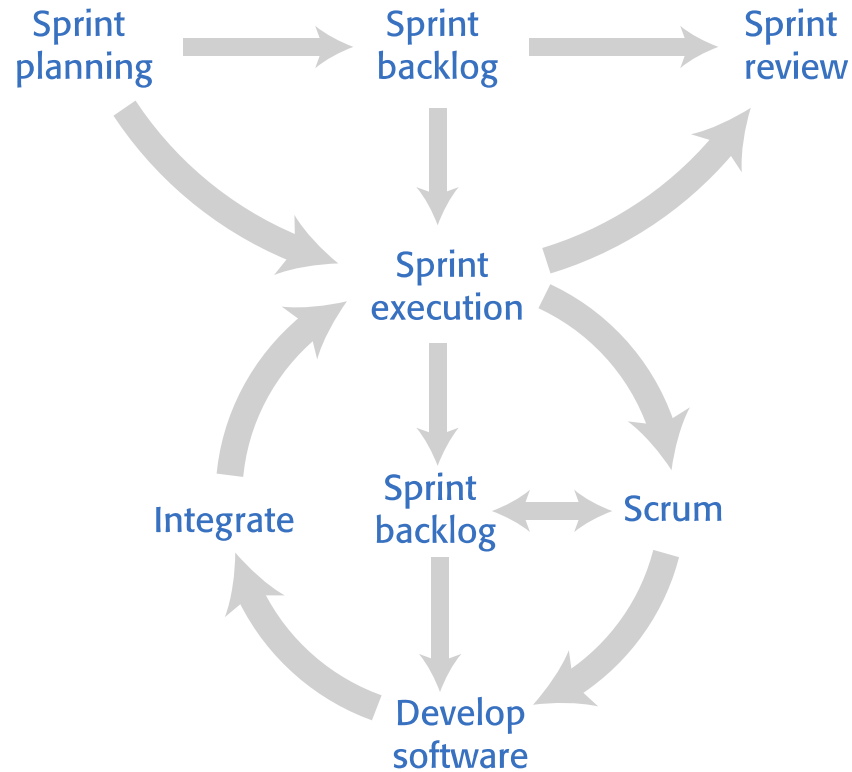
The team work to implement the sprint backlog items that have been chosen for that sprint. If it is impossible to complete all of the sprint backlog items, the sprint is not extended. The unfinished items are returned to the product backlog and queued for a future sprint.

Sprint activities (2 of 2)

- ***Sprint reviewing***

The work done in the sprint is reviewed by the team and (possibly) external stakeholders. The team reflect on what went well and what went wrong during the sprint with a view to improving their work process.

Figure 2.7

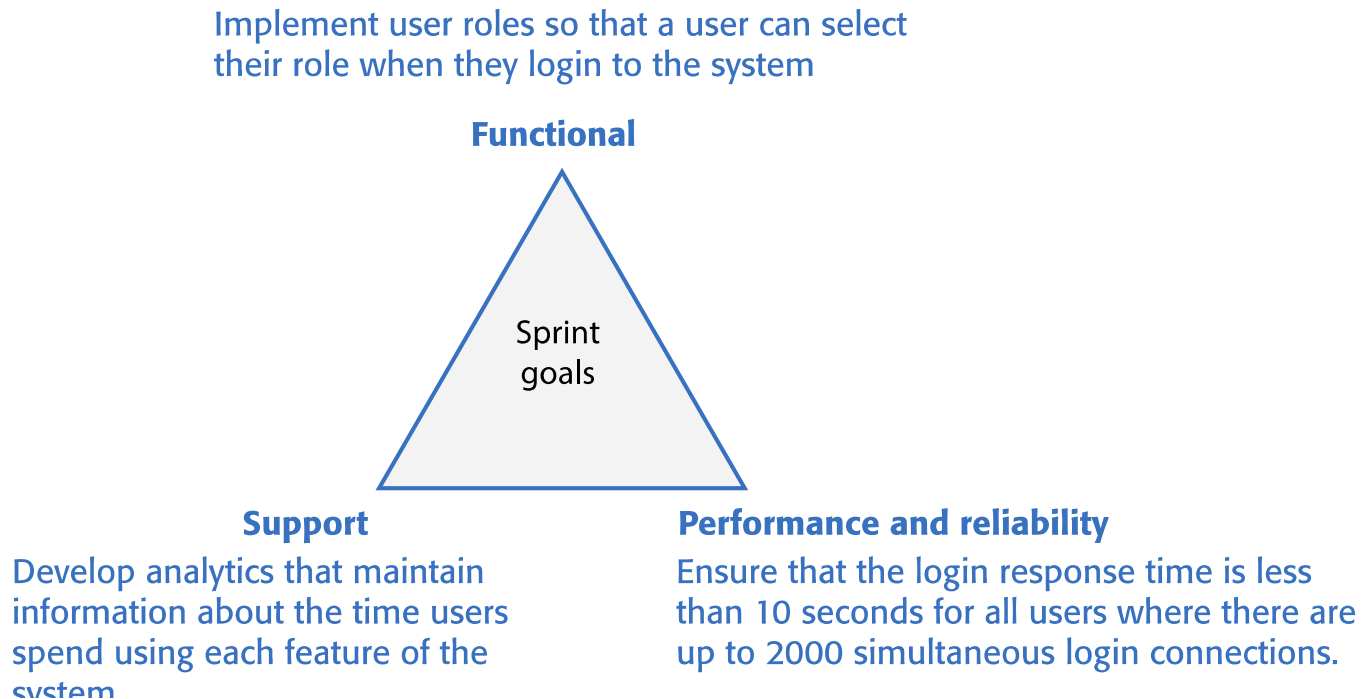


Sprint activities

Sprint planning (1 of 2)

- Establish an agreed sprint goal
 - Sprint goals may be focused on software functionality, support or performance and reliability,.
- Decide on the list of items from the product backlog that should be implemented
- Create a sprint backlog.
 - This is a more detailed version of the product backlog that records the work to be done during the sprint

Figure 2.8



Sprint goals

Sprint planning (2 of 2)

- In a sprint plan, the team decides which items in the product backlog should be implemented during that sprint.
 - Key inputs are the effort estimates associated with PBIs and the team's velocity
- The output of the sprint planning process is a sprint backlog.
 - The sprint backlog is a breakdown of PBIs to show the what is involved in implementing the PBIs chosen for that sprint.
- During a sprint, the team have daily meetings (scrums) to coordinate their work.

Table 2.8 Scrums

A scrum is a short, daily meeting that is usually held at the beginning of the day. During a scrum, all team members share information, describe their progress since the previous day's scrum, and present problems that have arisen and plans for the coming day. This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum meetings should be short and focused. To dissuade team members from getting involved in long discussions, scrums are sometimes organized as “stand-up” meetings where there are no chairs in the meeting room.

During a scrum, the sprint backlog is reviewed. Completed items are removed from it. New items may be added to the backlog as new information emerges. The team then decides who should work on sprint backlog items that day.

Agile activities

- Scrum does not suggest the technical agile activities that should be used. However, I think there are two practices that should always be used in a sprint.
- ***Test automation***
As far as possible, product testing should be automated. You should develop a suite of executable tests that can be run at any time.
- ***Continuous integration***
Whenever anyone makes changes to the software components they are developing, these components should be immediately integrated with other components to create a system. This system should then be tested to check for unanticipated component interaction problems.

Table 2.9 Code completeness checklist

State	Description
Reviewed	The code has been reviewed by another team member who has checked that it meets agreed coding standards, is understandable, includes appropriate comments, and has been refactored if necessary.
Unit tested	All unit tests have been run automatically and all tests have executed successfully.
Integrated	The code has been integrated with the project codebase and no integration errors have been reported.
Integration tested	All integration tests have been run automatically and all tests have been executed successfully.
Accepted	Acceptance tests have been run if appropriate and the Product Owner or the development team has confirmed that the product backlog item has been completed.

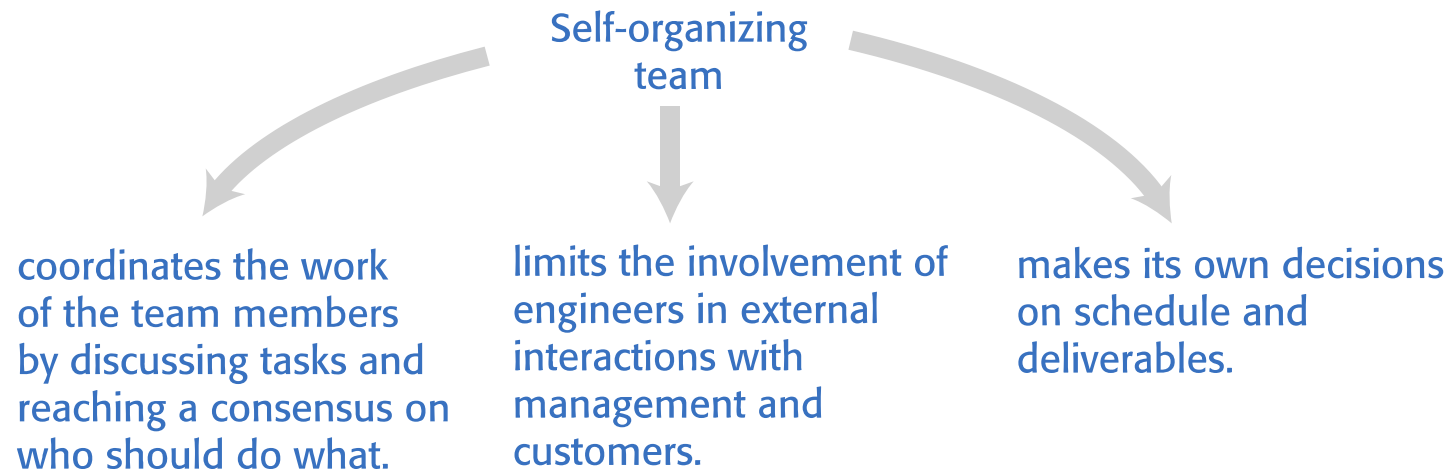
Sprint reviews (1 of 2)

- At the end of each sprint, there is a review meeting, which involves the whole team. This meeting:
 - reviews whether or not the sprint has met its goal.
 - sets out any new problems and issues that have emerged during the sprint.
 - is a way for a team to reflect on how they can improve the way they work.

Sprint reviews (2 of 2)

- The product owner has the ultimate authority to decide whether or not the goal of the sprint has been achieved. They should confirm that the implementation of the selected product backlog items is complete.
- The sprint review should include a process review, in which the team reflects on its own way of working and how Scrum has been used.
 - The aim is to identify ways to improve and to discuss how to use Scrum more productively.

Figure 2.9



Self-organizing teams

Team size and composition (1 of 2)

- The ideal Scrum team size is between 5 and 8 people.
 - Teams have to tackle diverse tasks and so usually require people with different skills, such as networking, user experience, database design and so on.
 - They usually involve people with different levels of experience.
 - A team of 5-8 people is large enough to be diverse yet small enough to communicate informally and effectively and to agree on the priorities of the team.

Team size and composition (2 of 2)

- The advantage of a self-organizing team is that it can be a cohesive team that can adapt to change.
 - Because the team rather than individuals take responsibility for the work, they can cope with people leaving and joining the team.
 - Good team communication means that team members inevitably learn something about each other's areas

Team coordination (1 of 2)

- The developers of Scrum assumed that teams would be co-located. They would work in the same room and could communicate informally.
 - Daily scrums mean that the team members know what's been done and what others are doing.

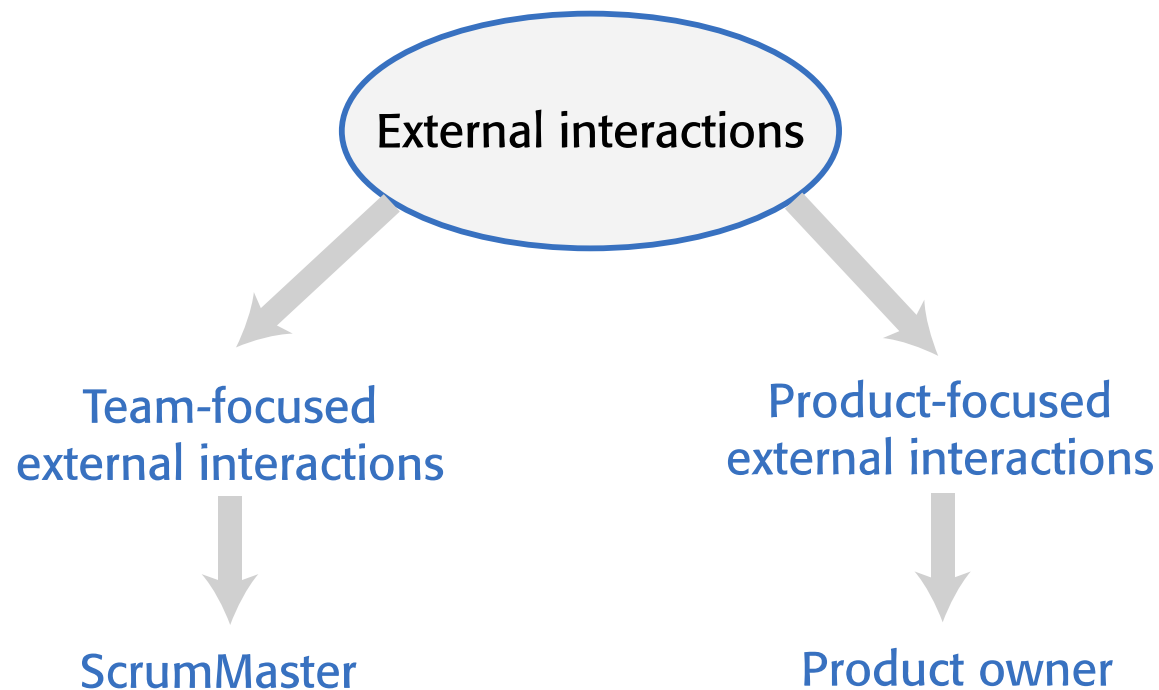
Team coordination (2 of 2)

- However, the use of daily scrums as a coordination mechanism is based on two assumptions that are not always correct:
 - Scrum assumes that the team will be made up of full-time workers who share a workspace. In reality, team members may be part-time and may work in different places. For a student project team, the team members may take different classes at different times.
 - Scrum assumes that all team members can attend a morning meeting to coordinate the work for the day. However, some team members may work flexible hours (e.g. because of childcare responsibilities) or may work on several projects at the same time.

External interactions

- External interactions are interactions that team members have with people outside of the team.
- In Scrum, the idea is that developers should focus on development and only the ScrumMaster and Product Owner should be involved in external interactions.
- The intention is that the team should be able to work on software development without external interference or distractions.

Figure 2.10



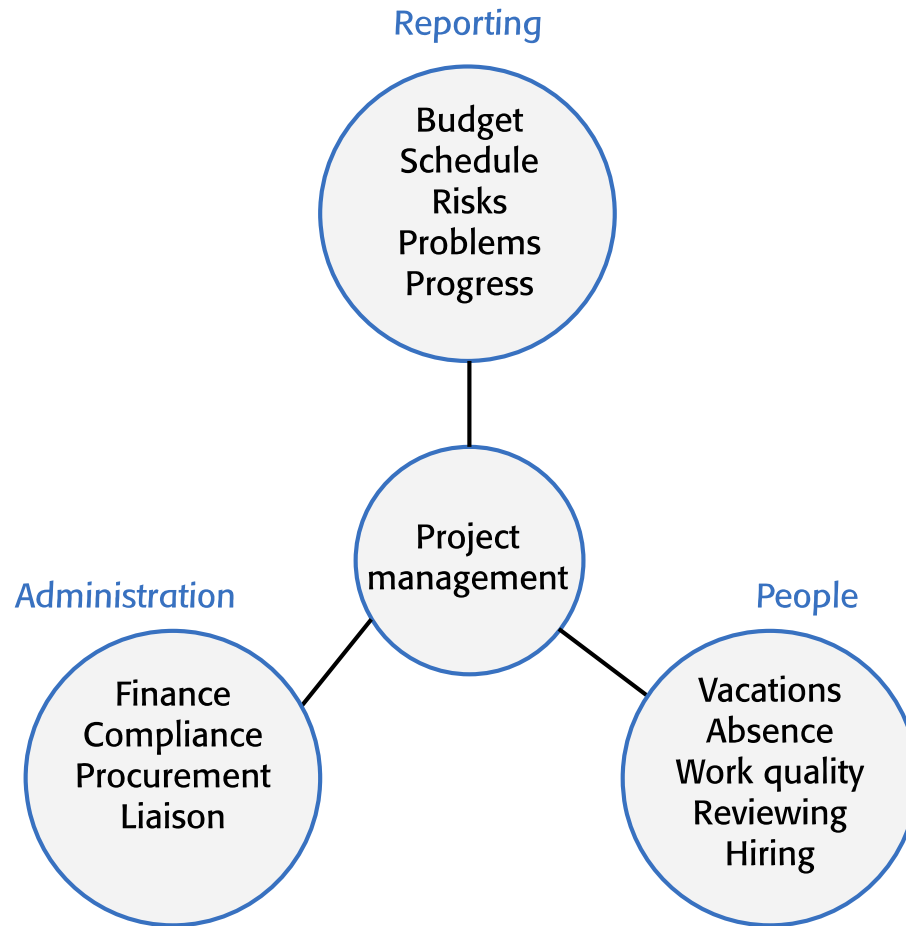
Project management (1 of 2)

- In all but the smallest product development companies, there is a need for development teams to report on progress to company management.
- A self-organizing team has to appoint someone to take on these responsibilities.
 - Because of the need to maintain continuity of communication with people outside of the group, rotating these activities around team members is not a viable approach.

Project management (2 of 2)

- The developers of Scrum did not envisage that the ScrumMaster should also have project management responsibilities.
 - In many companies, however, the ScrumMaster has to take on project management responsibilities.
 - They know the work going on and are in the best position to provide accurate information and project plans and progress.

Figure 2.11



Project management responsibilities

Key points 1 (1 of 2)

- The best way to develop software products is to use agile software engineering methods that are geared to rapid product development and delivery.
- Agile methods are based around iterative development and the minimization of overheads during the development process.
- Extreme programming (XP) is an influential agile method that introduced agile development practices such as user stories, test-first development and continuous integration. These are now mainstream software development activities.

Key points 1 (2 of 2)

- Scrum is an agile method that focuses on agile planning and management. Unlike XP, it does not define the engineering practices to be used. The development team may use any technical practices that they believe are appropriate for the product being developed.
- In Scrum, work to be done is maintained in a product backlog – a list of work items to be completed. Each increment of the software implements some of the work items from the product backlog.

Key points 2

- Sprints are fixed-time activities (usually 2–4 weeks) where a product increment is developed. Increments should be ‘potentially shippable’ i.e. they should not need further work before they are delivered.
- A self-organizing team is a development team that organizes the work to be done by discussion and agreement amongst team members.
- Scrum practices such as the product backlog, sprints and self-organizing teams can be used in any agile development process, even if other aspects of Scrum are not used.

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Table 2.3 Agile development principles

(1 of 2)

Principle	Description
Involve the customer	Involve customers closely with the software development team. Their role is to provide and prioritize new system requirements and to evaluate each increment of the system.
Embrace change	Expect the features of the product and the details of these features to change as the development team and the product manager learn more about the product. Adapt the software to cope with changes as they are made.
Develop and deliver Incrementally	Always develop software products in increments. Test and evaluate each increment as it is developed and feed back required changes to the development team.

Table 2.3 Agile development principles

(2 of 2)

Principle	Description
Maintain simplicity	Focus on simplicity in both the software being developed and the development process. Wherever possible, do what you can to eliminate complexity from the system.
Focus on people, not the development process	Trust the development team and do not expect everyone to always do things in the same way. Team members should be left to develop their own ways of working without being limited by prescriptive software processes.
	Description